

Programming Assignment #4

Due date: 2024 년 12 월 6 일 23 시 59 분

TA: 윤혜정 (hjyoon02@postech.ac.kr)

문의: PLMS - 게시판 - Q&A

문제 설명

과제 4 번은 STL 라이브러리의 컨테이너 중 하나인 List 를 직접 구현해보고, 직접 구현한 List 를 사용한 프로그램을 작성하는 것을 목표로 합니다. 아래 3 개의 문제를 차례로 해결하여 과제의 목표를 달성합니다.

- 문제 1 번에서는 List 에 대해 이해하기 위해 간단한 정수형 리스트인 IntList 클래스를 작성합니다.
- 문제 2 번에서는 여러 데이터 타입에 대해 확장할 수 있도록 Template 클래스인 List 를 작성합니다.
- 문제 3 번에서는 직접 만든 List 를 이용해 Student 클래스의 객체를 관리하는 프로그램을 작성합니다.

[Prob 1] Class IntList (50 점)

STL 중 자주 사용되는 컨테이너인 List 를 정수형에 대해서 구현합니다. 실제 STL 의 List 는 범용성을 가지기 위해 다양한 함수를 가지고 있고 데이터 구조가 최적화되어 있어 똑같이 구현하기는 어렵습니다. 따라서 본 과제에서는 그 중 리스트의 핵심 기능만을 구현하도록 합니다. 실제 STL list 의 설명은 [이 웹페이지](https://en.cppreference.com/reference/list/list/)에서 자세히 확인할 수 있습니다.

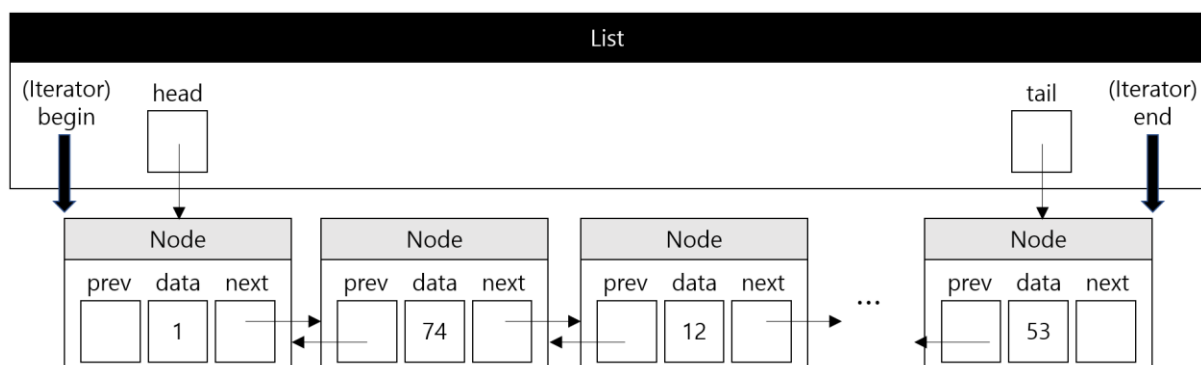
(<https://cplusplus.com/reference/list/list/>)

List 는 선형적 순서가 있는 데이터를 관리하기 위한 컨테이너입니다. 고정 크기를 가지는 배열과 달리 데이터의 삽입과 삭제가 자유롭기 때문에 자주 변하는 데이터를 다룰 때 적합합니다.

리스트는 이중 연결 리스트로 구현할 수 있습니다. 이를 위해 내부적으로 Node 구조체를 가지고 활용합니다. Node 객체는 자신의 데이터와 이전 Node 에 대한 포인터, 이후 Node 에 대한 포인터를 가지고 있어서 리스트 내의 데이터를 차례로 순회할 수 있게 합니다. 리스트는 Node 에 대한 포인터와 노드 순회를 위한 iterator 를 가짐으로써 데이터를 관리합니다. 이중 연결 리스트(Doubly linked list)에 대한 더 자세한 설명은 [위키피디아 페이지](https://en.wikipedia.org/wiki/Doubly_linked_list)를 참고하세요.

(https://en.wikipedia.org/wiki/Doubly_linked_list)

리스트의 데이터는 언제든지 삽입 또는 삭제되기 때문에 Node 는 동적 할당으로 생성 및 삭제되어야 합니다. 리스트는 Node 의 메모리를 순서대로 관리하기 위해 포인터와 iterator 를 사용합니다. 아래 그림은 리스트를 구현한 하나의 예시입니다. 아래 그림에서 리스트는 첫 노드(head)와 마지막 노드(tail)에 대한 포인터를 가지고, 사이에 있는 노드들은 노드의 양방향 포인터를 통해 연결됩니다. 또한, 리스트는 순회를 위해 노드의 처음(begin)과 끝(end)을 나타내는 iterator 를 가집니다.



문제 1 번에서 구현할 IntList 클래스는 private 멤버 변수로 Node 구조체를, public 멤버 변수로 iterator 클래스를 가지며, 리스트의 기능 구현을 위한 멤버 함수들을 가집니다. Node 및 iterator 를 제외한 멤버 변수는 필요에 따라 자율적으로 선언하고 설계할 수 있지만 반드시 private 으로 선언해야 합니다. Node 및 iterator 에 대해서는 생성자와 소멸자, 그리고 아래에 나열된 멤버 함수를 구현해야 합니다. List 의 멤버 함수는 아래의 명세를 따르도록 구현해야 합니다. 추가 멤버 함수는 private 으로 자유롭게 선언 및 구현할 수 있습니다.

IntList 멤버 변수

[Node 구조체]

- Node 구조체는 값을 담는 data 와 양 방향의 노드를 가리키는 포인터인 next, prev 를 멤버 변수로 갖습니다.
- 필요에 따라 생성자와 소멸자를 정의 및 구현합니다. 생성자는 인수를 받을 수도 있고, 기본 생성자를 사용할 수도 있습니다.

[iterator 클래스]

- iterator 클래스는 public 으로 선언된 중첩 클래스로, 외부에서 IntList::iterator it; 와 같이 변수를 선언할 수 있습니다. IntList 에만 특화된 반복자를 생성하여 제공함으로써 리스트의 순회를 더 쉽게 구현할 수 있게 해줍니다.
- iterator 의 생성자와 소멸자는 필요에 따라 정의 및 구현합니다. 생성자는 인수를 받을 수도 있고, 기본 생성자를 사용할 수도 있습니다.
- iterator 클래스는 현재 노드를 가리키는 노드 포인터 current 를 멤버 변수로 가지며, 현재 노드의 포인터를 반환하는 getNode 를 멤버 함수로 가집니다.
- iterator 의 멤버 함수로써 아래의 5 개의 연산자 오버로딩을 필수적으로 구현해야 합니다.
 - int& operator*(): 현재(current) 노드의 데이터를 반환합니다.
 - iterator& operator++(): iterator 를 한 칸 forward 방향으로 진행시킨 후 해당 iterator 를 반환합니다. 즉, 현재 노드의 다음 노드를 가리키는 iterator 를 반환합니다.
 - iterator& operator--(): iterator 를 한 칸 backward 방향으로 진행시킨 후 해당 iterator 를 반환합니다. 즉, 현재 노드의 이전 노드를 가리키는 iterator 를 반환합니다.

- `bool operator==(const iterator& other) const`: 두 `iterator` 를 비교하여 가리키는 노드가 같으면 `true` 를, 다르면 `false` 를 반환합니다.
- `bool operator!=(const iterator& other) const`: 두 `iterator` 를 비교하여 가리키는 노드가 다르면 `true` 를, 같으면 `false` 를 반환합니다.

[기타 멤버 변수]

- 리스트 구현에 필요한 기타 멤버 변수는 `private` 으로만 선언할 수 있으며, 필요에 따라 적절한 변수를 선언하여 활용합니다.

IntList 멤버 함수

[생성자 및 소멸자]

- `IntList()`: 빈 리스트를 생성합니다.
- `IntList(const IntList& l)`: 리스트를 인자로 받아 동일한 데이터를 갖는 새 리스트를 생성합니다 (복사 생성자). 깊은 복사를 구현하기 위해 노드를 모두 새로 동적 할당하고 데이터를 복사해야 합니다.
- `~IntList()`: 리스트에 있는 모든 노드들의 메모리를 할당 해제합니다.

[리스트 함수]

- `size_t size() const`: 리스트의 크기를 반환합니다. `size_t` 는 운영체제에 따라 적합한 크기를 갖는 `unsigned int` 자료형입니다.
- `bool empty() const`: 리스트가 비어 있는지 확인하여 비어 있다면 `true` 를, 아니면 `false` 를 반환합니다.
- `iterator begin()`: 리스트의 시작 반복자를 반환합니다.
- `iterator end()`: 리스트의 끝 반복자를 반환합니다. 리스트의 끝 노드를 가리키는 것이 아닌, 리스트의 끝 노드의 한 칸 뒤를 가리키는 반복자입니다. 그러나, 단순히 `nullptr` 를 가리키도록 설계한다면 `operator--` 를 이용해 리스트의 역방향 순회를 할 수 없기 때문에 `operator--` 를 한 번 실행했을 때 리스트의 끝 노드를 가리킬 수 있도록 디자인해야 합니다.
- `void push_front(const int& value)`: 리스트의 시작 부분에 `value` 라는 데이터를 가진 노드를 동적 할당하여 추가합니다.
- `void push_back(const int& value)`: 리스트의 끝 부분에 `value` 라는 데이터를 가진 노드를 동적 할당하여 추가합니다.
- `void pop_front()`: 리스트의 첫 노드를 제거합니다.
- `void pop_back()`: 리스트의 마지막 노드를 제거합니다.

- `void clear()`: 리스트를 비웁니다. 모든 노드의 메모리를 할당 해제하고, 리스트의 멤버 변수가 있다면 초기화하여 빈 리스트로 만듭니다.

[연산자 오버로딩]

- `int& operator[](size_t index)`: 리스트 중 `index` 번째 값의 레퍼런스를 반환합니다. 이 값은 수정할 수 있어야 합니다.
- `const int& operator[](size_t index) const`: 리스트 중 `index` 번째의 값의 레퍼런스를 읽기 전용으로 반환합니다. 이 값은 수정할 수 없습니다.
- `bool operator==(const IntList& other) const`: 두 리스트를 비교하여 같다면 `true` 를, 다르다면 `false` 를 반환합니다. 리스트가 같다는 것은 리스트의 노드 개수, 모든 데이터의 순서와 값이 정확히 같은 것을 의미합니다.
- `bool operator!=(const IntList& other) const`: 두 리스트를 비교하여 다르다면 `true` 를, 같다면 `false` 를 반환합니다.

[기타 멤버 함수]

- 필요에 따라 기타 멤버 함수를 `private` 으로 선언 및 구현할 수 있습니다.

[Prob 2] Template Class List (30 점)

문제 1 번에서 구현한 IntList 를 여러 자료 구조에 적용할 수 있도록 확장한 템플릿 클래스 List 를 구현합니다. private 멤버 변수는 자유롭게 선언할 수 있으나 public 멤버 변수로 iterator 클래스를 반드시 가져야 합니다. 또한, 아래의 명세를 따르는 멤버 함수를 가져야 하며, 추가로 필요한 멤버 함수는 private 으로 자유롭게 선언 및 구현할 수 있습니다. 기타 요구 사항은 문제 1 번과 동일합니다.

iterator 클래스 멤버 함수

- 생성자 및 소멸자 (생성자의 인수 및 오버로딩 개수는 자유)
- T& operator*()
- iterator& operator++()
- iterator& operator--()
- bool operator==(const iterator& other) const
- bool operator!=(const iterator& other) const

List 클래스 멤버 함수

[생성자 및 소멸자]

- List() (생성자)
- List(const List& l) (복사 생성자)
- ~List() (소멸자)

[리스트 함수]

- size_t size() const
- bool empty() const
- iterator begin()
- iterator end()
- void push_front(const T& value)
- void push_back(const T& value)
- void pop_front()
- void pop_back()
- void clear()

[연산자 오버로딩]

- T& operator[](size_t index)

- `const T& operator[](size_t index) const`
- `bool operator==(const List& other) const`
- `bool operator!=(const List& other) const`

[Prob 3] Score management program (20 점)

문제 2 번에서 직접 구현한 List 를 이용해 학생 성적을 관리하는 프로그램을 작성합니다. 성적 데이터는 학번, 국어성적, 영어성적, 수학성적 총 네 가지로 구성되어 있습니다. 프로그램 작성 시 학생 별 성적을 담는 Student 클래스를 만들고, Student 객체를 갖는 List 를 선언하여 활용해야 합니다. 데이터베이스에는 데이터를 추가하거나 삭제하는 기능이 있습니다. 데이터베이스 파일과 동작 파일이 주어지면 주어진 데이터베이스에서 주어진 동작을 실행한 결과를 파일로 출력합니다.

입력

- db.txt

학생들의 학번, 성적이 기입된 성적 데이터베이스 파일입니다. 최대 학생수는 50 명입니다. 각 줄마다 학번, 국어성적, 영어성적, 수학성적이 띄어쓰기로 구분되어 있습니다. 학번은 1000 이상 9999 이하의 정수, 각 과목의 성적은 0 이상 100 이하의 정수입니다.

[예시]

db.txt
1001 85 90 78
1004 90 88 85
1003 75 85 80

- action.txt

데이터베이스의 동작을 나타내는 파일입니다. 동작의 종류는 5 가지로, PUSH_FRONT, PUSH_BACK, POP_FRONT, POP_BACK, CLEAR 가 있습니다. 파일에서 각 줄이 하나의 동작을 나타내며, 각 동작의 설명과 파일 문자열 형식은 다음과 같습니다.

이름	설명	파일 문자열 형식
PUSH_FRONT	리스트의 첫 부분에 데이터 삽입	PUSH_FRONT {학번} {국어성적} {영어성적} {수학성적}
PUSH_BACK	리스트의 끝 부분에 데이터 삽입	PUSH_BACK {학번} {국어성적} {영어성적} {수학성적}
POP_FRONT	리스트의 첫 데이터 삭제	POP_FRONT

POP_BACK	리스트의 마지막 데이터 삭제	POP_BACK
CLEAR	리스트의 모든 데이터 삭제	CLEAR

[예시]

action.txt
POP_BACK CLEAR PUSH_FRONT 1002 88 67 92 PUSH_BACK 1005 88 67 92 POP_FRONT PUSH_FRONT 1001 85 90 78

출력

- output.txt
db.txt 와 같은 양식으로 동작이 적용된 데이터베이스를 출력합니다.
위의 두 파일이 입력일 때, 예시 출력은 다음과 같습니다.

output.txt
1001 85 90 78 1005 88 67 92

output.txt 가 없다면 새로 만들고, 파일이 이미 있다면 출력 내용으로 덮어씹습니다.
데이터베이스가 비어 있다면 내용이 없는 output.txt 가 만들어져야 합니다.

주의 사항

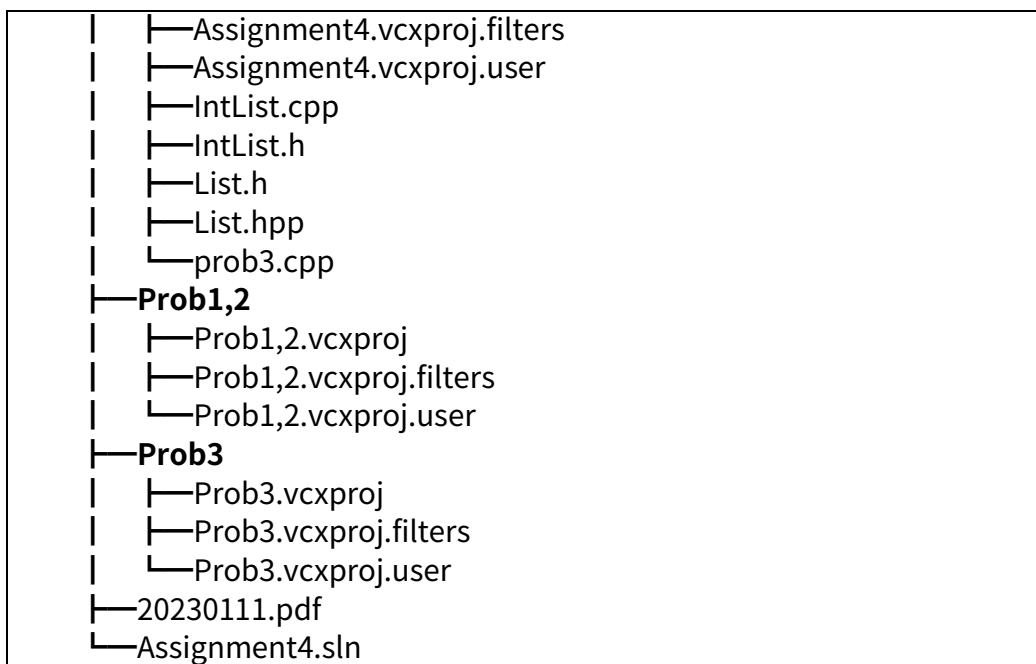
- 문제 3 번에서는 잘못된 입력에 대한 예외 처리를 고려하지 않아도 됩니다. 모든 파일은 양식에 맞게 주어지며, 데이터가 없을 때 POP_FRONT, POP_BACK 을 하는 경우는 없습니다.
- 반드시 문제 2 번에서 직접 구현한 List 를 사용해 구현해야 합니다. STL 컨테이너인 list, vector 등은 사용하면 안 됩니다.

제출 방법

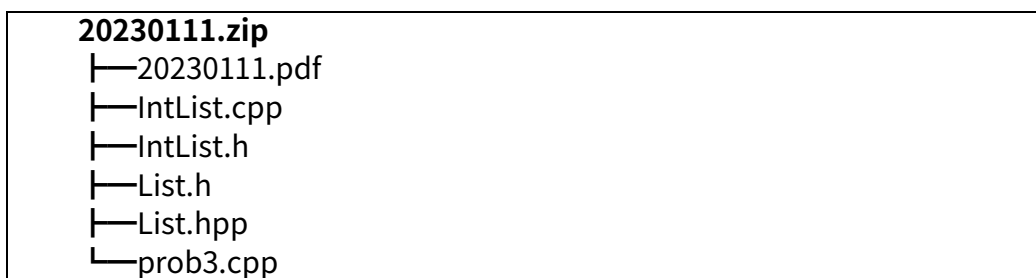
- 이번 과제는 Baseline 코드를 제공하므로 각 문제에 해당하는 파일을 수정 및 완성하여 문제의 요구 사항을 구현합니다. IntList.h 와 IntList.cpp 는 문제 1 번, List.h 와 List.cpp 는 문제 2 번, prob3.cpp 는 문제 3 번에 해당합니다. test.cpp 는 문제 1 번과 2 번의 테스트를 위한 예시 코드로, 채점에 포함되지 않습니다.
- Baseline 코드는 문제 조건을 만족하는 범위 내에서 자유롭게 수정이 가능하지만 파일을 추가하거나 삭제할 수는 없습니다. STL 컨테이너(list, vector 등)는 사용할 수 없으며, 그 외의 필요한 라이브러리는 자유롭게 import 하여 사용할 수 있습니다.
- 예시로 제공하는 Visual Studio 솔루션에는 두 개의 프로젝트가 있습니다. Prob1,2 프로젝트는 IntList 와 List 를 포함하며, test.cpp 의 코드를 컴파일 및 실행하여 작성한 IntList 와 List 를 테스트해볼 수 있습니다. Prob3 프로젝트는 List 를 포함하며, prob3.cpp 에서 List 를 사용한 프로그램을 작성하여 컴파일 및 실행할 수 있습니다. 솔루션 탐색기에서 프로젝트를 우클릭하여 “시작 프로젝트로 설정”을 클릭하면 컴파일 및 실행할 프로젝트를 선택할 수 있습니다.
- 문제 1 번과 2 번에서 구현한 IntList 와 List 는 test.cpp 에서 반드시 정상적으로 import 되고 기능해야 합니다. 따라서 test.cpp 코드가 컴파일이 되는 것을 꼭 확인한 후 제출하시기 바랍니다. test.cpp 에 예시로 제공하는 테스트 외의 다른 테스트를 직접 만들어 IntList 와 List 의 기능들을 확인해보기를 추천합니다. test.cpp 파일은 채점에 포함되지 않으므로 제출 시 삭제해야 합니다.
- 제출 파일에는 IntList.h, IntList.cpp, List.h, List.cpp, prob3.cpp 만을 포함합니다. test.cpp, db.txt, action.txt, output.txt 등은 삭제하고, 디버그 폴더 및 불필요한 파일(e.g., x64, Debug, bin, *.obj, *.o, *.exe, a.out)도 반드시 삭제한 후 업로드 합니다.
- 보고서는 {학번}.pdf 로 저장하여 과제 폴더 안에 위치시킨 후 제출합니다. 보고서는 문제 3 번에 대해서만 학기 초에 공지한 Report Form 을 따라 작성합니다. 문제 1 번, 문제 2 번에 대해서는 보고서를 작성하는 대신 코드 내 주석을 자세히 작성해주세요. 과제 문서의 12-13 페이지에 기술된 주석 작성 요령을 참고하시기 바랍니다.
- 작성한 코드 및 보고서는 {학번}.zip 으로 압축하여 제출합니다. 아래의 제출 예시를 참고하세요.

- Windows 제출 예시 (20230111 은 자신의 학번으로 대체)

```
20230111.zip
├─Assignment4
│   └─Assignment4.vcxproj
```



■ Mac 제출 예시



주의사항

- AssignmentReadMe 와 Report Form 파일을 충분히 숙지하여 과제를 진행하세요.
- 문제 1, 2 번은 보고서 대신 주석을 자세하게 작성하세요. 문제 3 번은 보고서를 Report Form 의 양식대로 작성하세요.
- 채점은 Windows Visual Studio 2022 환경에서 이루어집니다.
- 문제 1 번과 2 번의 기능 점수는 test.cpp 와 형식이 같지만 내용이 다른 비공개 테스트케이스를 이용해 채점합니다.
- 문제 3 번의 기능 점수는 db.txt, action.txt 와 형식이 같지만 내용이 다른 비공개 테스트케이스를 이용해 출력 파일을 확인하여 채점합니다.
- 과제 관련 질문은 메일이 아닌 Q&A 게시판을 이용해 주세요. 질문하기 전에 과제 문서를 여러 번 정독하고, Q&A 게시판에 동일한 질문이 없는지 확인하시기 바랍니다.

주석 작성 요령

문제 1 번과 2 번은 보고서를 작성하지 않고, 구현에 관한 모든 내용을 코드 주석으로 자세하게 작성해야 합니다. 아래의 작성 요령을 따라 본인이 작성한 모든 멤버 변수, 함수 또는 멤버 함수에 다음과 같은 주석을 작성해 주세요. 주석을 작성하지 않은 함수에 대해서는 기능 및 설계 점수를 받을 수 없습니다.

이번 과제의 주석은 Javadoc 에서 사용하는 Document comment 형식을 따르도록 합니다. 이렇게 정형화된 함수 주석을 사용하면 Javadoc 문서 생성기를 이용해 소스 코드로부터 바로 HTML 형식의 API 문서를 생성할 수 있어 편리합니다. 그래서 STL 컨테이너를 배포하는 등 공식적인 함수 주석이 필요한 경우 Javadoc 에서 사용하는 문서 주석을 표준으로 사용하기도 합니다. (참고: <https://en.wikipedia.org/wiki/Javadoc>)

이번 과제에서는 문서 변환을 할 필요는 없기 때문에 형식과 문법은 신경 쓰지 않고, 아래에 기술된 형식을 따라 보고서를 작성하듯 주석을 작성하면 됩니다.

함수 주석의 일반적인 형식은 매개변수(param), 리턴값(return), 메모(note), 경고(warning), 버그(bug) 등으로 구성됩니다. 아래는 그 예시입니다.

```
/**
 * This first line is a brief description.
 *
 * The rest of the lines are a more detailed description of the
 * function that outlines what it does and anything interesting about
 * how it does it.
 *
 * @param x Description of the first parameter.
 * @param y Description of the second parameter.
 * @param z Description of the third parameter.
 *
 * @return This is where you describe the possible return values.
 * If the function is void then there must not be a @return markup
 * element in this doc box! (Don't document something that does not exist.)
 *
 * @note This is how you can add a note about the function that you implemented
 * And please list any references you used in your implementation.
 *
 * @warning This is how you can add an optional warning to a user of the
 * function suitable for noting things like 'This function is not thread
 * safe' and so on.
 *
 * @bug This is how you can add an optional description of a known bug in the
 * function such as: This only works for positive values of z.
 *****/
int f(int x, int y, int z) {
    ...
}
```

@param 과 @return 의 경우 단순히 타입만 나열하는데 그치지 않고, 해당 매개변수와 리턴 값의 목적과 의미를 설명해야 합니다. 참고 예시는 다음과 같습니다.

```
/**
 * Searches an array of Student objects for a specified name.
 *
 * @param student_array An array of Student objects to search.
 * @param search_name A name for which to search.
 *
 * @return The index of the array element that contains searchName or
 *         -1 if the name is not found.
 *
 * @note Uses the binary search algorithm, so student_array must be sorted
 *       in ascending order by name.
 *****/
int search_for_name(const Student student_array[], const string& search_name) {
    ...
}
```

함수의 설명은 이미 이 문서의 명세에 나와 있으므로 단순히 설명을 옮겨 적는 것 이상의 정보가 포함되어야 합니다. 예를 들면 매개변수와 리턴값의 의미, 구현 혹은 사용 시 주의 사항과 코드를 왜 그렇게 작성했는가에 대해 본인의 생각을 위의 양식에 따라 작성합니다. (한국어로 작성해도 됩니다.)

간단한 함수의 경우 자세한 설명을 생략할 수 있고, 매개변수나 리턴 값이 없을 경우 그 또한 생략 가능합니다. @note @warning @bug 도 필요시에만 추가 작성합니다.

함수가 아니라 특별히 선언한 멤버 변수와 같이 구체적 설명이 필요한 라인 혹은 섹션에는 다음과 같이 주석을 작성합니다. 이때, 변수와 함수 이름을 의미 있게 작성한다면 많은 주석을 작성하지 않고 아낄 수 있습니다.

```
average = (double) sum / count;    // Calculate the program average.

// Calculate the area of the triangle: semi is semiperimeter;
// s1, s2, and s3 are sides. (multi line)
area = sqrt(semi * (semi - s1) * (semi - s2) * (semi - s3));
```

클래스 멤버 함수의 경우 이러한 주석은 h 파일이 아닌 cpp (또는 hpp) 파일 구현부 위에 작성하도록 합니다. 구현 시 참조한 reference 나 토론한 다른 학생이 있다면 그에 대한 내용도 작성합니다.